# Day 121: DBSCAN From Scratch: Density-Based Clustering on Non-Convex Data

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 3 | Complete study guide

Watch: Clustering with DBSCAN, Clearly Explained!!! (StatQuest with Josh Starmer) | DBSCAN Explanation and Visualization (TheDataPost)

Script: `day121_dbscan.py` · every number in this guide is copied by program from `day121_run_output.txt`, the real output of that script.

DBSCAN finds clusters as dense regions of any shape and leaves sparse points as noise; it needs no k. Written from scratch, it matches scikit-learn exactly, separates two moons and two rings perfectly (1.0000, where k-means scores 0.2475 and 0.0039), and, given only the positions of 20,640 Californian census districts, finds the state's cities: Los Angeles (7176 districts), the Bay Area split by the bay itself, San Diego, Sacramento and Fresno. Two honest failures: the usual k-distance 'knee' rule chose a poor eps on all three datasets, and no single eps suits clusters of different densities.

## 0. Where today fits

**DEFINITION — start here, no prior knowledge assumed**

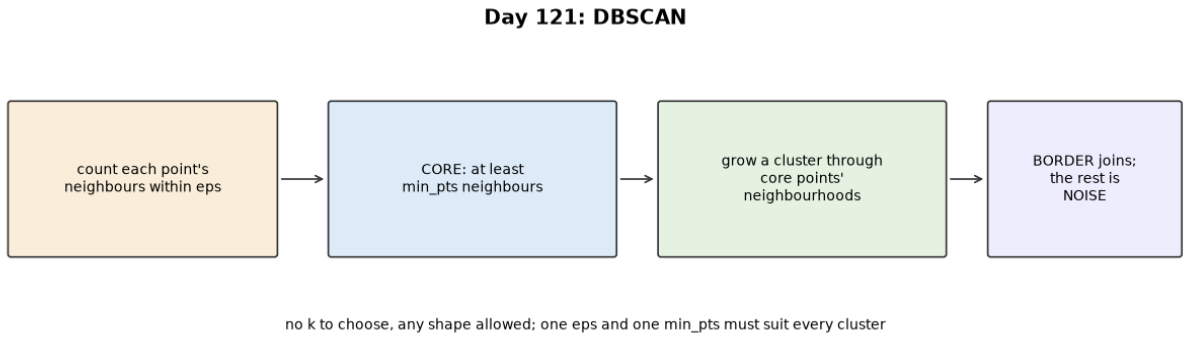
**DBSCAN** (density-based spatial clustering of applications with noise) has two settings: a radius **eps** and a count **min_pts**. A point with at least min_pts points within eps (itself included) is a **core** point. Clusters grow from core points through their neighbourhoods; a non-core point within eps of a core point is a **border** point and joins that cluster; everything else is **noise**.

Unlike k-means (Day 119), the number of clusters is found, not chosen, and clusters can take any shape.

**REAL-WORLD SCENARIO**

Looking at the earth at night from space: cities are bright, densely packed regions of any shape, joined along coasts and valleys, and scattered farms are noise. Nobody needs to say in advance how many cities there are.

**SYSTEM DIAGRAM**




*Count neighbours, mark core points, grow clusters through them; border points join, the rest is noise.*

### Today's part for the week's build

`day121_exercise.py` is the neighbourhoods, DBSCAN and the k-distance, checked by hand on eleven points. Its real output:

**ACTUAL OUTPUT**

```
eleven points, eps 0.6, min_pts 3: labels [0, 0, 0, 0, 1, 1, 1, -1, -1, -1, -1]
core points: [0.5, 1.0, 5.4]; by hand: 0.5, 1 and 5.4 (three points within 0.6 each)
noise: [9.0, 12.0, 12.3, 20.0]; by hand: 9, 12, 12.3 and 20
2nd-nearest distance (itself first) of 0, 9 and 12: [0.5, 3.0, 0.3]; by hand: 0.5, 3.0 and 0.3
eps 0.1: every point noise True; eps 100: one cluster True; min_pts 1: no noise True
```

Data: two moons and a wide-plus-tight pair (Day 119's code and seed) and two rings, made on purpose so the true groups are known exactly; 20,640 California census districts from scikit-learn's California housing dataset (downloaded once, cached outside the repository), of which only latitude and longitude are clustered; and the digits. scikit-learn's DBSCAN is an outside check.

### How to read each section

| Step | What it gives you |
|---|---|
| DEFINITION | What the idea is, in plain words. Start here. |
| REAL-WORLD SCENARIO | An everyday situation where the same idea shows up. |
| MATHEMATICS | The formula, when the section introduces one. |
| SPECIFICATION | The exact setup of the experiment. |
| IMPLEMENTATION | The real code from the script. |
| ACTUAL OUTPUT | What the script really printed, copied unchanged. |
| What it means | How to read the numbers, and what they do and do not show. |
| Why choose this? / Why NOT this? | When the idea is the right tool, and what it costs. |

## 1. By hand: eleven points

**DEFINITION — start here, no prior knowledge assumed**

Points on a line, eps = 0.6, min_pts = 3: count each point's neighbours, mark the core points, and grow.

**REAL-WORLD SCENARIO**

Houses along a road: a house with two neighbours within 600 m is in a village; a house next to a village house is on its edge; the rest are farms.

**MATHEMATICS**

$$N(x) = \{y : |x - y| \le eps\}; \quad x \text{ is core if } |N(x)| \ge min_pts; \quad \text{clusters} = \text{core points joined through their neighbourhoods}$$

**IMPLEMENTATION**

```
def neighbours(X, eps, chunk=1000):
    out = []
    for s in range(0, len(X), chunk):
        D = sqdist(X[s:s + chunk], X)
        out += [np.where(row <= eps * eps)[0] for row in D]
    return out
def dbscan(X, eps, min_pts):
    N = neighbours(X, eps); core = np.array([len(nb) >= min_pts for nb in N])
    lab = np.full(len(X), NOISE); c = 0
    for i in range(len(X)):
        if lab[i] != NOISE or not core[i]:
            continue
        lab[i] = c; stack = [i]
        while stack:
            j = stack.pop()
            if core[j]:
                for v in N[j]:
                    if lab[v] == NOISE:
                        lab[v] = c; stack.append(v)
        c += 1
    return lab, core
```

**ACTUAL OUTPUT**

```
  point   0.0: 2 within 0.6 (itself included) -> border label 0
  point   0.5: 3 within 0.6 (itself included) -> core   label 0
  point   1.0: 3 within 0.6 (itself included) -> core   label 0
  point   1.5: 2 within 0.6 (itself included) -> border label 0
  point   5.0: 2 within 0.6 (itself included) -> border label 1
  point   5.4: 3 within 0.6 (itself included) -> core   label 1
  point   5.8: 2 within 0.6 (itself included) -> border label 1
  point   9.0: 1 within 0.6 (itself included) -> noise  label -1
  point  12.0: 2 within 0.6 (itself included) -> noise  label -1
  point  12.3: 2 within 0.6 (itself included) -> noise  label -1
  point  20.0: 1 within 0.6 (itself included) -> noise  label -1
by hand: 0.5, 1 and 5.4 have 3 points within 0.6, so they are core; 0, 1.5 and 5.8 are within 0.6
    of a core point, so they
join its cluster as border points; 9, 12, 12.3 and 20 are noise (12 and 12.3 are close, but two
    points are fewer than min_pts)
Actual output from this lesson's run.
```

### What it means

Three core points (0.5, 1 and 5.4) start two clusters; 0, 1.5, 5 and 5.8 are border points; 9, 12, 12.3 and 20 are noise. The pair 12 and 12.3 is the instructive case: close together, but two points are fewer than min_pts, so neither is core and neither can start a cluster.

**WHY CHOOSE THIS?**

A rule simple enough to apply by hand.

**WHY NOT THIS? (TRADEOFFS)**

Everything hangs on two numbers chosen by the user.

## 2. Correctness: against scikit-learn

**DEFINITION — start here, no prior knowledge assumed**

The same settings on three datasets; core points and labels compared one by one.

**REAL-WORLD SCENARIO**

Checking a hand count of a crowd against a turnstile.

ACTUAL OUTPUT

```
  two moons             (eps 0.2, min_pts 5): core points identical True; labels identical True;
      clusters 2
  rings                 (eps 0.8, min_pts 5): core points identical True; labels identical True;
      clusters 2
  California districts  (eps 5.0, min_pts 50): core points identical True; labels identical True;
      clusters 29
Actual output from this lesson's run.
```

### What it means

Identical core points and identical labels on all three, including the 29 clusters of California. Both versions expand clusters in the same order, so even border points within reach of two clusters go to the same one.

**WHY CHOOSE THIS?**

Exact agreement, not just similar scores.

**WHY NOT THIS? (TRADEOFFS)**

scikit-learn uses a faster neighbour search; this version computes every distance.

## 3. Shapes k-means cannot represent

**DEFINITION — start here, no prior knowledge assumed**

Two interleaved moons and a ring inside a ring, with DBSCAN and with Day 119's k-means.

**REAL-WORLD SCENARIO**

Following a river by walking from bank to bank, rather than drawing a straight line across the map.

ACTUAL OUTPUT

```
  two moons : DBSCAN (eps 0.2, min_pts 5) ARI 1.0000, 2 clusters, noise 0.000; k-means ARI 0.2475
  rings     : DBSCAN (eps 0.8, min_pts 5) ARI 1.0000, 2 clusters, noise 0.000; k-means ARI 0.0039
DBSCAN follows the shape through chains of dense neighbourhoods; k-means draws one straight
    boundary
Actual output from this lesson's run.
```

### What it means

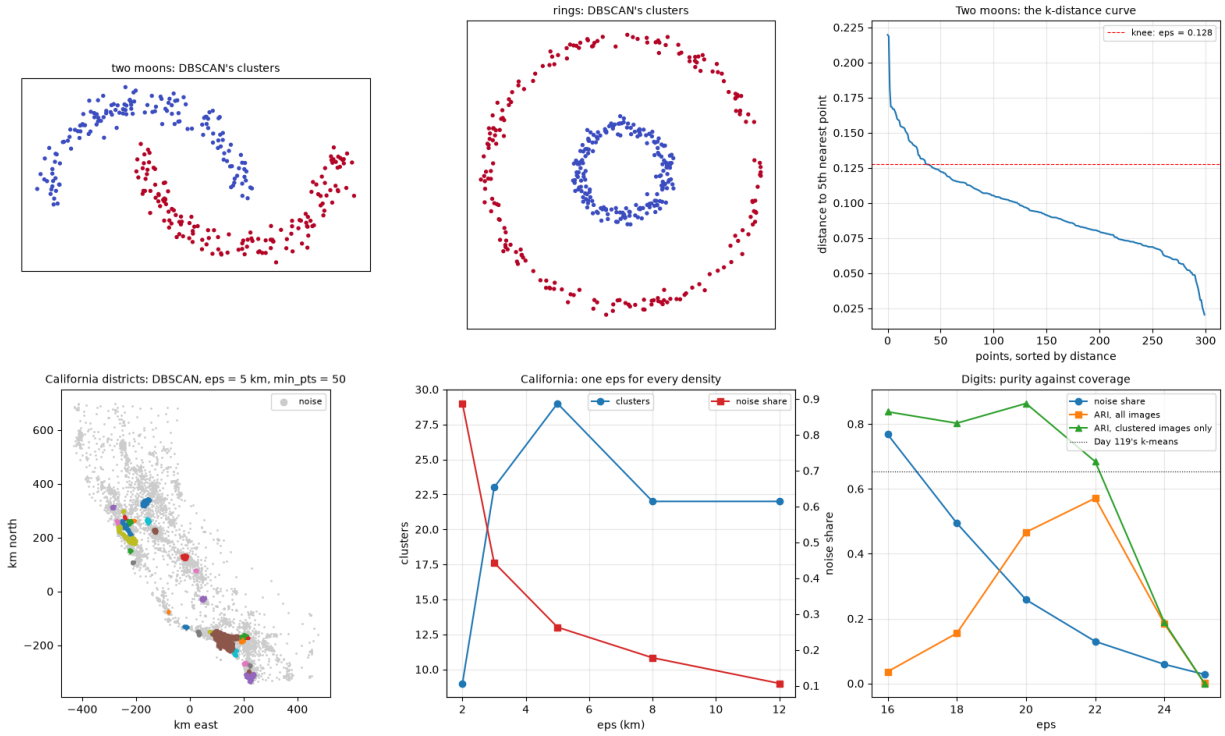
DBSCAN recovers both shapes exactly (1.0000). k-means cannot: one straight boundary cuts the moons (0.2475) and slices both rings in half (0.0039, no better than chance). DBSCAN never measures a cluster's shape; it only asks whether the points are connected through dense neighbourhoods.

**WHY CHOOSE THIS?**

Any shape, no k, and outliers labelled as noise.

**WHY NOT THIS? (TRADEOFFS)**

Both settings were chosen knowing the answer; section 4 asks how to choose them without it.




*Top: DBSCAN on the moons and the rings, and the k-distance curve. Bottom: California (grey = noise), the eps scan on California, and digits.*

## 4. Choosing eps: the k-distance curve

**DEFINITION — start here, no prior knowledge assumed**

The usual guide: sort every point's distance to its min_pts-th nearest point, and take eps at the knee of that curve. Here the knee is found by Day 119's elbow rule.

**REAL-WORLD SCENARIO**

Choosing a search radius by asking how far each house is from its fifth-nearest neighbour.

**IMPLEMENTATION**

```
def k_distance(X, k, chunk=1000):
    out = []
    for s in range(0, len(X), chunk):
        D = np.maximum(sqdist(X[s:s + chunk], X), 0)
        out.append(np.sqrt(np.partition(D, k - 1, axis=1)[:, k - 1]))
    return np.concatenate(out)
def knee_eps(X, k):
    kd = np.sort(k_distance(X, k))[::-1]
    i = d119.elbow(list(range(len(kd))), kd)
    return float(kd[i]), kd
```

**ACTUAL OUTPUT**

```
each point's distance to its 5th nearest point (itself first), sorted; the knee of that curve is
    at eps = 0.128
     eps  clusters   noise      ARI
     0.1        16   0.130   0.1378
    0.15         2   0.003   0.9934
     0.2         2   0.000   1.0000
     0.3         2   0.000   1.0000
     0.5         1   0.000   0.0000
   0.128         6   0.027   0.4083
too small: the moons break into fragments and noise; too large: they merge into one
Actual output from this lesson's run.
```

### What it means

The moons are found for eps from 0.15 to 0.3; below that they break into fragments (16 clusters at 0.1), above it they merge into one. The knee rule chose 0.128, just below the working range, and scored only 0.4083. The same rule suggests 21.53 km for California and 25.16 for the digits, both far too large (sections 5 and 7). The curve's bend is a hint, not an answer.

**WHY CHOOSE THIS?**

A quick, data-driven starting point.

**WHY NOT THIS? (TRADEOFFS)**

Here it failed on all three datasets; checking a range of eps is safer.

## 5. Real data: where Californians live

**DEFINITION — start here, no prior knowledge assumed**

Only the latitude and longitude of 20,640 census districts, converted to kilometres; eps = 5 km and min_pts = 50, chosen after trying eps from 2 to 8 km and min_pts from 20 to 100. The median house value of each district (from the same dataset) is used afterwards to describe the clusters.

**REAL-WORLD SCENARIO**

Drawing city limits from a map of rooftops.

**IMPLEMENTATION**

```
def california():
    d = fetch_california_housing(); lat, lon = d.data[:, 6], d.data[:, 7]
    km = np.c_[(lon - lon.mean()) * 111.32 * np.cos(np.radians(lat.mean())), (lat - lat.mean()) *
        110.57]
    return km, lat, lon, d.target
def cities_in(lab, c, lat, lon):
    out = []
    for name, (a, b) in CITIES.items():
        i = int(np.argmin(np.hypot((lat - a) * 110.57, (lon - b) * 111.32 *
            np.cos(np.radians(a)))))
        out += [name] if lab[i] == c else []
    return ", ".join(out) if out else "(none listed)"
```

ACTUAL OUTPUT

```
29 clusters; 0.263 of districts are noise (rural, or between towns); the knee rule would suggest eps = 21.53
    km
    size  centre (lat, lon)    median house value  listed cities inside
    7176  (33.98, -118.17)               207,850  Los Angeles
    1952  (37.51, -122.17)               298,350  San Francisco, San Jose
    1124  (37.78, -122.20)               187,150  Oakland
    1086  (32.74, -117.10)               157,950  San Diego
     667  (38.60, -121.39)               118,000  Sacramento
     341  (36.78, -119.77)                73,000  Fresno
     299  (34.11, -117.32)                93,300  (none listed)
  noise districts: median house value 127,900; districts in any cluster: 195,400
nobody told DBSCAN where the cities are: the clusters are where districts are densely packed
Actual output from this lesson's run.
```

### What it means

With no idea where the cities are, DBSCAN finds them: Los Angeles (7176 districts), San Diego, Sacramento, Fresno and San Bernardino. The Bay Area comes out as two clusters, San Francisco with San Jose (1952) and Oakland (1124): the bay itself, with no districts on the water, keeps them apart at this density. A quarter of districts (0.263) are noise, and they are cheaper: median house value 127,900 against 195,400 inside clusters.

WHY CHOOSE THIS?

Finds dense regions of any shape on real data, with no k.

WHY NOT THIS? (TRADEOFFS)

The settings were chosen by trying several; another choice gives another map (section 6).

## 6. One eps for every density

DEFINITION — start here, no prior knowledge assumed

The same California data at five values of eps; and the wide-plus-tight pair, at 21 settings.

REAL-WORLD SCENARIO

One rule for 'a town' (fifty houses within 5 km) applied to both Manhattan and the Mojave.

ACTUAL OUTPUT

```
California, min_pts = 50: the same data at different eps
   eps (km)  clusters   noise  largest  listed cities in the largest
        2.0         9   0.888      971  (none listed)
        3.0        23   0.443     6297  Los Angeles
        5.0        29   0.263     7176  Los Angeles
        8.0        22   0.178     8317  Los Angeles
       12.0        22   0.106     8652  Los Angeles
wide + tight (450 spread-out points beside 50 tight ones): the best of 21 settings is eps 0.3,
    min_pts 10: ARI 0.5548, 2 clusters, noise 0.916
an eps small enough to separate the tight group leaves most of the wide one as noise; one large
    enough for the wide one swallows the tight one
Actual output from this lesson's run.
```

### What it means

At 2.0 km, 0.888 of districts are noise: only the densest city cores qualify. At 12.0 km the noise falls to 0.106 but the largest cluster grows to 8652 as Los Angeles absorbs its neighbours. No setting gives both. The wide-plus-tight pair shows the same problem cleanly: the best of 21 settings scores 0.5548 and leaves 0.916 of the points as noise. Variants such as OPTICS and HDBSCAN exist to handle several densities.

**WHY CHOOSE THIS?**

One radius is simple to explain.

**WHY NOT THIS? (TRADEOFFS)**

Clusters of different densities cannot all be found with it.

## 7. High dimensions: digits

**DEFINITION — start here, no prior knowledge assumed**

DBSCAN on the 64 pixels of each digit image, min_pts = 5, at several eps.

**REAL-WORLD SCENARIO**

Looking for crowds in a city where everyone stands about equally far from everyone else.

**ACTUAL OUTPUT**

```
the knee rule suggests eps = 25.16; Day 119's k-means scored ARI 0.6525 on the same images
    eps  clusters   noise  ARI, all  ARI, clustered only
   16.0        27   0.768    0.0364               0.8376
   18.0        37   0.494    0.1548               0.8024
   20.0        25   0.258    0.4664               0.8633
   22.0        13   0.130    0.5711               0.6842
   24.0         8   0.059    0.1841               0.1882
  25.16         3   0.028    0.0006               0.0003
('ARI, all' counts noise as one more group; 'clustered only' judges just the images DBSCAN placed)
Actual output from this lesson's run.
```

### What it means

There is a trade-off between purity and coverage. At eps 16 the clusters are fairly pure (0.8376 agreement among clustered images) but 0.768 of images are noise. The best overall score, 0.5711 at eps 22, is still below Day 119's k-means (0.6525), and by eps 24 the digits merge (0.1841). In 64 dimensions distances bunch together (Day 115), so 'dense' and 'sparse' are hard to tell apart.

**WHY CHOOSE THIS?**

Points it does cluster are clustered well.

**WHY NOT THIS? (TRADEOFFS)**

In high dimensions it leaves many points as noise, or merges everything.

## Interview preparation — technical questions

**Q: How does DBSCAN work?**

A: Core points have at least min_pts neighbours within eps; clusters grow through core points; the rest is border or noise.

**Q: When would you choose DBSCAN over k-means?**

A: Non-round shapes, unknown k, outliers: moons 1.0000 against 0.2475.

**Q: How do you choose eps?**

A: Start from the k-distance curve, then check a range; the knee rule alone chose badly on all three datasets here.

**Q: What happens with very different densities?**

A: No single eps fits: California noise went from 0.888 to 0.106 as eps grew, while LA swallowed its neighbours.

**Q: Is DBSCAN deterministic?**

A: The core points and noise are; a border point reachable from two clusters goes to the one found first.

## Interview preparation — theoretical questions

**Q: Why can DBSCAN find any shape?**

A: It never measures a cluster's shape, only whether points connect through dense neighbourhoods.

**Q: What is a border point?**

A: Not core itself, but within eps of a core point.

**Q: Why does DBSCAN struggle in high dimensions?**

A: Distances bunch together, so density stops separating groups: digits never beat k-means here.

**Q: What does min_pts control?**

A: How dense a region must be: two close points (12 and 12.3) were noise with min_pts = 3.

## Key takeaway and what's next

> **KEY TAKEAWAY**
>
> DBSCAN replaces 'how many clusters?' with 'how dense is a cluster?'. With a sensible radius it found moons, rings and the cities of California, Bay and all, with no k and no labels. Its single radius is also its limit: one eps cannot fit dense city cores and sprawling suburbs at once, and the textbook knee rule was no reliable guide.

- Moons and rings: 1.0000 (k-means 0.2475 and 0.0039).
- California: 29 clusters at 5 km; noise districts' median value 127,900 against 195,400.
- Knee rule: 0.128 on the moons (ARI 0.4083); 21.53 km on California.
- Digits: best 0.5711, below k-means' 0.6525.

### Suggested watch order, if starting from zero

- 1. Clustering with DBSCAN, Clearly Explained!!! (StatQuest with Josh Starmer) — DBSCAN's core, border and noise points.
- 2. DBSCAN Explanation and Visualization (TheDataPost) — the algorithm step by step, with an interactive visualisation.

### Day 122 preview

- **Gaussian mixture models and the EM algorithm from scratch**: soft clusters, each an ellipse with its own shape and size.